

SaaSCommand 360

Student Assignment | SaaS Product, Revenue & Customer Success Analytics

Student Name	
Student ID	
Team Members	
Submission Date	
Mentor	

1. Project Objective

Build a SaaS command center combining product usage, subscriptions, billing, payments and support to monitor customer health, forecast revenue and automate retention/expansion workflows.

2. Business Problem

SaaS businesses need a unified view of product adoption and revenue. A customer can be paying but disengaging, or highly engaged but underutilized on a plan.

3. Expected Product Outcome

The student must deliver a working application, not a notebook collection. The final demo should prove that data enters the platform, is transformed into governed analytical models, produces business intelligence, triggers an automated action, and becomes visible through an API and frontend.

4. Data Sources & Dataset Design

Source	Required fields
Accounts	account_id, industry, size, region, owner
Users	user_id, account_id, role, created_at
Product events	event_id, user_id, feature, timestamp
Subscriptions	subscription_id, account_id, plan, start_date, status
Invoices	invoice_id, account_id, amount, due_date, status
Support	ticket_id, account_id, severity, created_at, resolved_at

5. End-to-End Architecture

Source systems → ingestion → Kafka/event bus → Bronze data lake → Silver cleansing → PySpark/SQL transformations → cloud warehouse → dbt analytical marts → analytics/ML → automation → FastAPI → React/Next.js → cloud deployment.

Recommended technologies: Python, SQL, Kafka, PySpark, GCS/S3, BigQuery/Snowflake, dbt, Airflow, FastAPI, React/Next.js, Docker and GitHub Actions. Equivalent technologies are allowed.

6. Live Streaming Requirement

Product-usage, subscription and payment events via Kafka; live customer-health metrics.

Mandatory streaming evidence: show the event producer, Kafka topic(s), consumer/processor, persistent analytical output and frontend/API update. Include retry handling, duplicate-event strategy and basic monitoring.

7. Data Lake & Data Warehouse

fact_usage, fact_billing, fact_support, fact_subscription + customer/account, user, product, plan and date dimensions.

Required engineering practices: raw immutable ingestion, schema validation, incremental loading, partitioning/clustering where appropriate, primary/business keys, referential integrity and audit columns.

8. Analytical Layer

Create a governed semantic/analytical layer with reusable SQL models. Required outputs include executive KPIs, operational metrics, drill-down views, trend analysis, segmentation and exception analysis.

Minimum KPI	Expected analytical question
MRR	How is mrr changing, what drives it, and what action should the business take?
ARR	How is arr changing, what drives it, and what action should the business take?
Logo churn	How is logo churn changing, what drives it, and what action should the business take?
Revenue churn	How is revenue churn changing, what drives it, and what action should the business take?
Expansion	How is expansion changing, what drives it, and what action should the business take?
Contraction	How is contraction changing, what drives it, and what action should the business take?
DAU/MAU	How is dau/mau changing, what drives it, and what action should the business take?
Feature adoption	How is feature adoption changing, what drives it, and what action should the business take?
Support SLA	How is support sla changing, what drives it, and what action should the business take?

9. Intelligence / ML

Churn-risk scoring, expansion-opportunity scoring, usage anomaly detection and revenue forecasting.

Every model must include a business objective, feature definitions, train/validation/test approach where applicable, evaluation metrics, threshold selection and a short explanation of limitations.

10. Automation & Decision Engine

Implement at least the following domain rules:

- Health drop → CSM alert
- Payment failure → billing workflow
- Adoption threshold → upsell recommendation
- Support deterioration → customer-success escalation

At least two automated actions must send a notification through email, Slack or Teams. Every alert must have a timestamp, severity, owner/status, trigger reason and audit history.

11. Frontend Application

- Customer 360
- Revenue dashboard
- Product analytics
- Customer health
- Expansion center
- Alerts

Minimum UX: login, navigation, KPI cards, charts, filters, drill-down, live/refresh indicator, alert center, loading/error states and responsive layout.

12. Backend API

- GET /api/customers/{id}/health
- GET /api/revenue
- GET /api/product/usage

- GET /api/churn-risk
- POST /api/customer-action
- GET /api/dashboard
- GET /api/events/live
- GET /api/alerts
- GET /api/forecasts
- GET /api/risk
- POST /api/alerts/{id}/acknowledge
- GET /api/data-quality

Use FastAPI or an equivalent framework. Include request validation, authentication, authorization, error handling, logging and OpenAPI documentation.

13. Orchestration & Automation

Use Airflow or an equivalent scheduler for batch ingestion, transformations, quality checks, model refresh and report/alert workflows. Implement retries, failure notifications and an operational run log.

14. Data Quality & Observability

- Null, duplicate and uniqueness checks
- Referential-integrity checks
- Valid-range and business-rule checks
- Freshness and volume checks
- Pipeline success/failure monitoring
- Alert/audit logging
- Basic API/application logging
- Documented recovery procedure for a failed pipeline

15. Cloud & DevOps

- Deploy the application and core services to a cloud platform.
- Use Docker for repeatable environments.
- Use GitHub with meaningful commit history and branching.
- Implement CI/CD for testing and deployment.
- Use environment variables or cloud secret management for credentials.
- Document architecture, deployment steps and estimated monthly cloud cost.
- Add basic monitoring and logs.

16. Security & Governance

- Authentication and role-based authorization
- No credentials committed to Git
- Least-privilege service accounts/roles
- Data masking or synthetic data for sensitive fields
- Audit trail for important automated actions
- Input validation and API rate/abuse considerations

17. Mandatory Deliverables

Deliverable	Status
GitHub repository	Mandatory
Architecture diagram	Mandatory
ERD / star schema	Mandatory
Data dictionary	Mandatory
SQL scripts	Mandatory
dbt project	Mandatory
Airflow workflows	Mandatory
Kafka streaming implementation	Mandatory
Data-quality tests	Mandatory
ML/forecasting/risk module	Mandatory
Automation/notification module	Mandatory
FastAPI backend	Mandatory
React/Next.js frontend	Mandatory
Docker configuration	Mandatory
CI/CD workflow	Mandatory
Cloud deployment	Mandatory
Test cases	Mandatory
Final presentation	Mandatory
Demo video	Recommended

18. Six-Week Implementation Plan

Week	Outcome
1	Business analysis, requirements, dataset generation/acquisition, architecture, GitHub setup
2	Ingestion, Bronze/Silver layers, warehouse foundation and dimensional model
3	Kafka streaming, PySpark processing, dbt marts and data-quality tests
4	Analytics layer, ML/forecasting/risk model and automation engine
5	FastAPI, frontend, authentication, roles and application integration
6	Cloud deployment, Docker, CI/CD, testing, documentation and final demo

19. Interview Pitch

Student should be able to explain: "I built SaaSCommand 360, an end-to-end saas product, revenue & customer success analytics platform. It ingests live events, stores governed data in a warehouse, creates business analytics, applies intelligence to predict/detect risk, automatically triggers actions, exposes APIs, and runs as a cloud application."

20. Technical Interview Questions

- How do you calculate SaaS retention?
- How do you separate seasonality from churn?
- How do you define customer health?
- How would you handle millions of product events?

- How do you validate churn predictions?
- Why did you choose your warehouse schema?
- How do you prevent duplicate processing?
- How do you handle late or out-of-order events?
- How would you scale the solution by 10x?
- How would you reduce cloud cost?
- How do you test the end-to-end pipeline?
- What would you monitor in production?

21. Evaluation Rubric — 100 Marks

Area	Marks
Business problem & solution design	10
SQL & analytical reasoning	10
Data modeling & warehouse	10
Data engineering & orchestration	10
Streaming / real-time processing	10
Analytics / ML / forecasting	10
Automation & decisioning	10
Backend & frontend application	10
Cloud / Docker / CI/CD	10
Documentation / presentation / interview	10

22. Final Success Criteria

The project is complete only when a reviewer can see a live/simulated event enter the system, observe it being processed, verify its warehouse/analytical representation, see an insight or model result, trigger an automated business action, and view the result through the deployed application.